605.486 – Android Development

Module 17
Files

Application Permissions

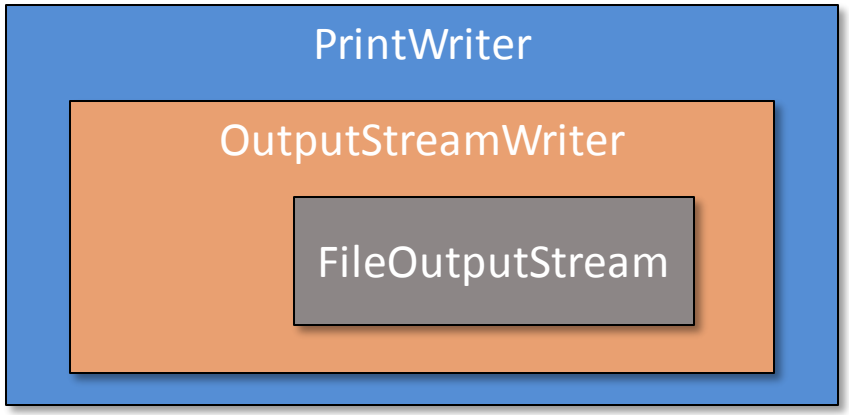
- Each application is assigned a user id when installed on a device
  - The id can vary across devices
  - Unique per application
  - Prevents apps from seeing each other's data
  - Prevents users from seeing app data
    - Can make debugging on a device painful
    - Unless phone is rooted!
      - Or emulator

Files

- Modes
  - MODE_PRIVATE
  - MODE_WORLD_READABLE
  - MODE_WORLD_WRITEABLE
- Location
  - System
  - External Storage
    - Application
    - Shared Directories
- Proper Exception Handling

Example 1 - Writing PRIVATE

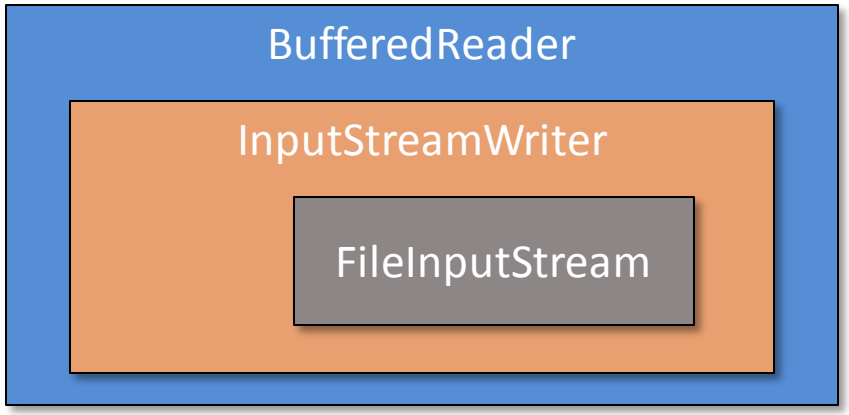
- openFileOutput – returns FileOutputStream
- Decorate with PrintWriter



- Write content
- Close (NOTE: bad placement in this example)

Example 2 - Reading PRIVATE

- openFileInput – returns FileInputStream
- Decorate with BufferedReader



- Read content
- Close (NOTE: bad placement in this example)

Example 3 - Try read from other app

- Same read structure
  - createPackageContext("com.javadude.files1",0)
    - Reference other application's context to call open

- Fails!

Example 4 - WORLD_READABLE

- Change access in app1 to WORLD_READABLE
  - Note deprecation warning!
- Re-run
- Try to read from app2

Example 5 - External Storage

- Check Environment.getExternalStorageState()
- getExternalFilesDir(null) – gets application dir
  - NOT PRIVATE
  - Deleted when app is uninstalled
- Create and write file

- Try without permission
- Try with WRITE_EXTERNAL_STORAGE permission

Example 6 - Shared Directories

- Environment.getExternalStoragePublicDirectory(
  Environment.DIRECTORY_MUSIC);

- (rest stays the same)

Example 7 - Proper Exception Handling

- Messy…
- Step-by-step walkthrough

Example 8 - Using Template Method

- AutoFileCloser
  - Constructor contains exception logic
  - Constructor calls doWork() hook
  - Subclasses override doWork()
  - doWork() registers streams to close via watch()

- Note: Java 7 try-with-resources similar
  - Cannot use in Android (yet)…